



Qualcomm Technologies, Inc.

eAI to QNN Migration Guide

80-VN500-41 Rev. AC

November 4, 2025

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:16 GMT
hyungchul.won@lge.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to:
DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	June 2025	Initial release
AB	October 2025	Editorial update
AC	November 2025	Miscellaneous updates throughout

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:16 GMT
hyungchul.won@lge.com

Contents

1 Introduction	5
1.1 Purpose.....	5
1.2 Conventions.....	5
1.3 Technical assistance	5
2 Prerequisites	6
2.1 Software Package	6
3 Qualcomm AI Engine Direct overview	7
3.1 Software architecture.....	7
3.1.1 Device.....	7
3.1.2 Backend.....	7
3.1.3 Context.....	7
3.1.4 Graph.....	8
3.2 Workflow migration from eAI to QNN	9
3.2.1 Offline LPAI model generation.....	9
3.2.2 LPAI model validation via qnn-net-run	10
3.2.3 Deployment on target using QNN APIs.....	14
3.3 Model conversion.....	15
3.4 Model preparation	18
3.4.1 Generation of QNN model library.....	18
3.4.2 Generation of QNN context binary for LPAI Backend.....	18
4 QNN API Descriptions and Call Sequences	21
4.1 QNN APIs usage guidelines.....	22
4.2 Data structures and enumerations.....	23
4.2.1 QnnContext_Config_t	23
4.2.2 QnnContext_ConfigOption_t	23
4.2.3 QnnBackend_Property_t.....	23
4.2.4 QnnLpaiDevice_DeviceInfoExtension_t.....	23
4.2.5 QnnLpaiGraph_Mem_t.....	24
4.2.6 QnnLpaiMem_MemType_t.....	24
4.2.7 QnnLpaiGraph_SetConfigOption_t.....	24
4.2.8 QnnGraph_Config_t	25
4.2.9 QnnLpaiGraph_CustomConfig_t.....	25
4.2.10 QnnLpaiBackend_BufferAlignmentReq_t.....	25
4.2.11 QnnLpaiBackend_GetPropertyOption_t.....	25
4.2.12 QnnLpaiGraph_CustomProperty_t	26

4.2.13 QnnLpaiGraph_CoreAffinity_t	26
4.2.14 QnnLpaiGraph_CoreAffinityType_t.....	26
4.2.15 QnnLpaiGraph_PerfCfg_t.....	27
4.2.16 QnnLpaiGraph_ClientPerfType_t.....	27
4.2.17 Qnn_ApiVersion_t.....	27
4.2.18 Qnn_Version_t.....	28
4.3 QNN APIs Call Flow	28
4.3.1 Initialization.....	28
4.3.2 Execution.....	31
4.3.3 Deinitialization	31
5 Reference example	32
5.1 Build and Run CAPI based QNN example on Target (ADSP).....	32
5.1.1 Compiling CAPI example in Hexagon SDK	32
5.1.2 Build the SoundTriggerTestApp from the LA build	33
5.1.3 Update resourcemanager.xml file and ACDB files	33
5.2 CAPI example overview	33
5.2.1 Data structures and enumerations	33
5.2.2 Initialization.....	33
5.2.3 Execute.....	35
5.2.4 Deinitialization	35
6 FAQs	36
A References	38
6.2 Related documents.....	38
6.3 Acronyms and terms.....	38

1 Introduction

1.1 Purpose

This document provides an overview of the QNN framework and the QNN APIs used to integrate QNN models in the QNN-LPAI backend on Snapdragon chipsets.

It details the components and functionalities of the QNN framework, explaining how to use QNN APIs to enable seamless integration of QNN models and guiding developers through the technical aspects of deploying neural network models efficiently on QTI's platform.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `#include`.

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

Shading indicates content that has been added or changed in this revision of the document.

Correct setup is important for a reliable and accurate calibration procedure. Steps that the user **must** do are essential, and failure to do this may cause calibration to be inaccurate or fail. Steps that the user **should** do are recommended, but not essential. For example, the user **must** measure the target according to the instructions in this document (this is essential). The target **should** have a light-colored background, but this is not essential.

1.3 Technical assistance

For assistance, submit a case to Qualcomm Technologies, Inc. (QTI) at <https://createpoint.qti.qualcomm.com/>.

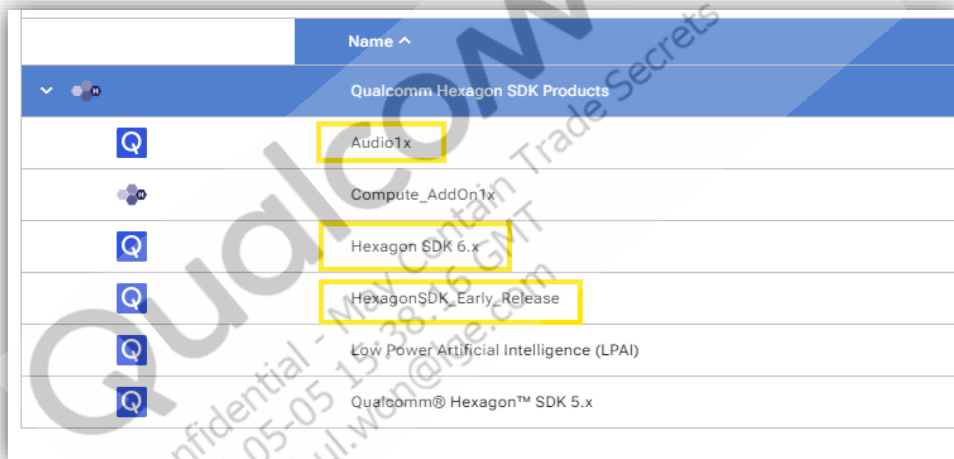
If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

2 Prerequisites

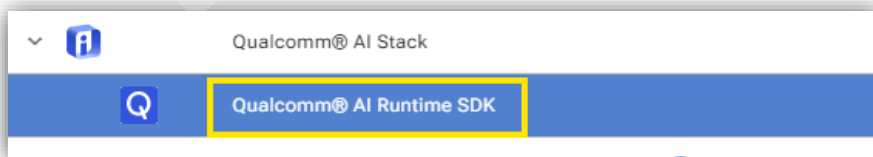
2.1 Software Package

The following tools are required to be installed:

1. Qualcomm Package Manager (QPM) 1.0.114 or later, downloaded from CreatePoint.
2. Install the following software products from QPM available under “Qualcomm Hexagon SDK Product Suite”
 - Hexagon SDK – 6.2.x onwards
 - Audio add-on – 3.x.x
 - Hexagon SDK early release add-on – 5.1.0.0 onwards



3. Install the Qualcomm AI Runtime SDK from QPM under “Qualcomm AI Stack”.



3 Qualcomm AI Engine Direct overview

Qualcomm AI Engine Direct is Qualcomm Technologies Inc. (QTI)'s software architecture for AI/ML use cases on QTI chipsets and AI acceleration cores.

The Qualcomm AI Engine Direct architecture is designed to provide a unified API and modular and extensible per-accelerator libraries, which form a reusable basis for full stack AI solutions, both for QTI's own and third-party frameworks.

3.1 Software architecture

The Qualcomm AI Engine Direct API and the associated software stack provides all the constructs required by an application to construct, optimize, and execute network models on the preferred hardware accelerator core.

3.1.1 Device

- A device is the software abstraction of a hardware accelerator platform.
- It provides all the constructs required to associate the preferred hardware accelerator resources for the execution of user-composed graphs.
- A platform is broken down into potentially multiple devices, and these devices may have multiple cores.

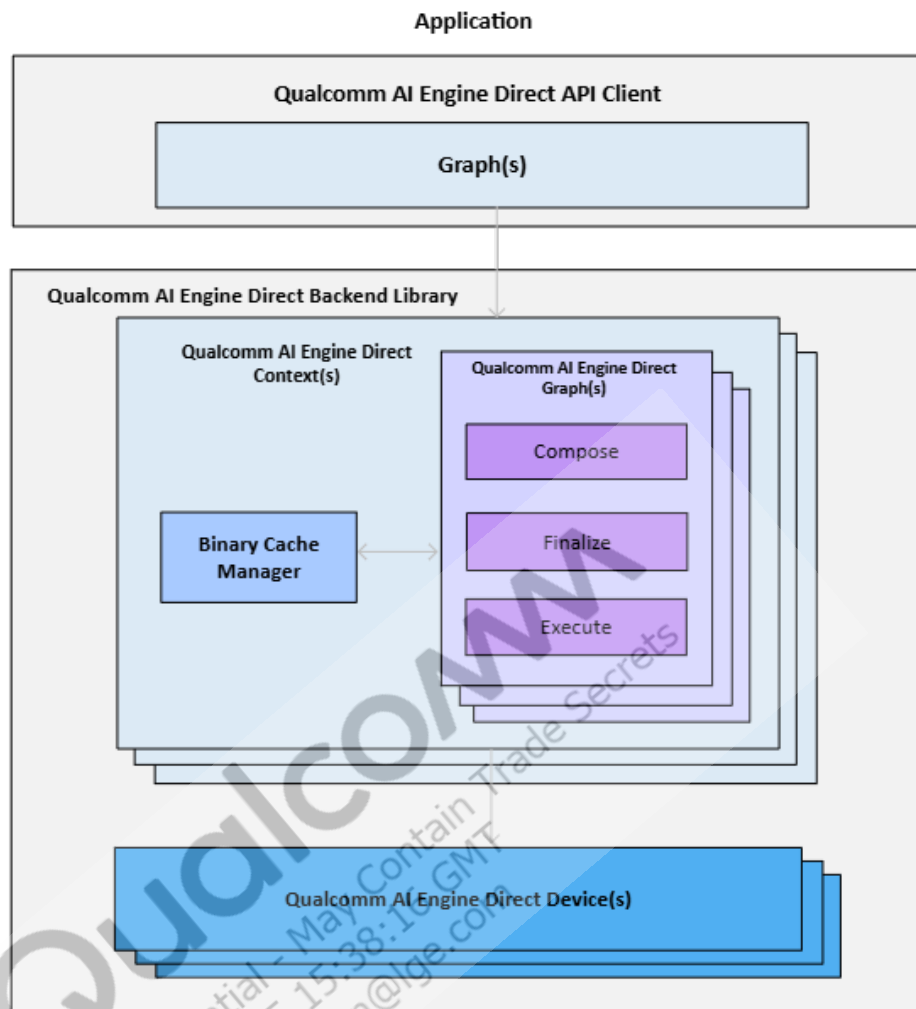
3.1.2 Backend

The backend is a top-level API component that hosts and manages most of the backend resources required for graph composition and execution, including an operation registry that stores all available operations.

3.1.3 Context

- A construct that represents all Qualcomm AI Engine Direct components required to sustain a user application.
- It hosts networks provided by the user and allows constructed entities to be cached into serialized objects for future use. Additionally, it enables interoperability between multiple graphs by providing a shareable memory space in which tensors can be exchanged between graphs.

The following diagrams provides a high level view of Qualcomm AI Engine Direct components.

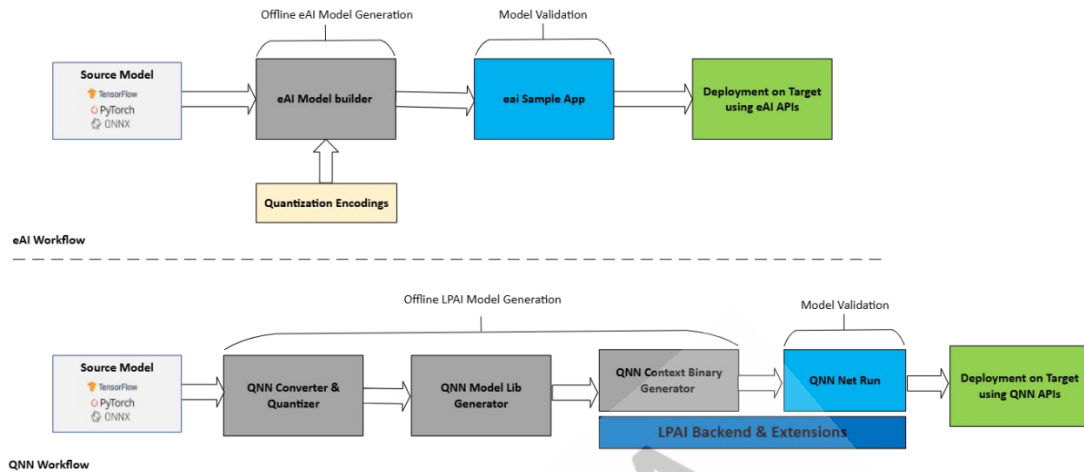


3.1.4 Graph

- A graph is a direct way of representing a loadable network model.
- It consists of nodes that represent operations and tensors that interconnect them to compose a directed acyclic graph.
- The Qualcomm AI Engine Direct graph construct supports APIs that perform initialization, optimization, and execution of network models.

3.2 Workflow migration from eAI to QNN

The following diagram shows the eAI vs. QNN workflow.



The QNN workflow is divided into the following stages:

1. Offline LPAI model generation
2. LPAI model validation via qnn-net-run
3. Deployment on target using QNN APIs

3.2.1 Offline LPAI model generation

The following is the QNN workflow for offline LPAI model generation:

1. Convert the source model into a QNN-compatible format using QNN Converter tools (see Section 3.3)
 - The QNN Converter tool also takes care of quantization if the user has provided a representative input which depicts the behavior of the model. However, the user can override it by providing a quantization encoding file as well.
2. Generate a model library (see Section 3.4.1)
3. Generate the context binary for LPAI backend (see Section 3.4.2)

3.2.2 LPAI model validation via qnn-net-run

There are three ways to validate a QNN-LPAI model via qnn-net-run:

- LPAI x86 Linux/Windows simulation – This mode supports both offline model preparation and direct execution using qnn-net-run.
 - Direct execution – In this mode, qnn-net-run internally prepares a graph and executes it on x86. Usages:

```
{QNN_SDK_ROOT}/bin/x86_64-<Platform>/qnn-net-run
--backend libQnnLpai.so
--input_list input_list.txt
--model <libQnnModel>.so
--config_file <qnn_config.json>
```

NOTE: <libQnnModel.so> must be built for the x86 platform (either Windows or Linux).

Based on the platform, select the appropriate qnn-net-run from the respective folders (either x86_64-linux-clang or x86_64-windows-msvc). See Section 3.4.1 for more details.

- Using serialized context binary. Usages:

```
{QNN_SDK_ROOT}/bin/x86_64-<platform>/qnn-net-run
--backend libQnnLpai.so
--input_list input_list.txt
--retrieve_context <lpai_model>.bin
--config_file <qnn_config.json>
```

NOTE: <lpai_model>.bin must be built for the x86 platform (either Windows or Linux).

Based on the platform, select the appropriate qnn-net-run from the respective folders (either x86_64-linux-clang or x86_64-windows-msvc). See Section 3.4.2 for more details.

- LPAI ARM backend execution via FastRPC – This mode is exclusively for offline-prepared graphs. Steps:

- a. Set environments:

```
export QNN_TARGET_ARCH=aarch64-android
export HW_VER=v6
```

- b. Prepare qnn_config.json and lpai_config.conf files:

- qnn_config.json:

```
{
  "backend_extensions": {
    "shared_library_path": "/data/local/tmp/libQnnLpaiNetRunExtensions.so",
    "config_file_path": "./lpai_config.conf"
  }
}
```

```

    }
}

```

- **lpai_config.conf:**

```

{
  "lpai_backend":
  {
    "target_env": "adsp",
    "enable_hw_ver": "v6"
  },
  "lpai_graph":
  {
    "execute":
    {
      "fps": 30,
      "ftrt_ratio": 10,
      "client_type": "non_real_time",
      "affinity": "soft",
      "core_selection": 0
    }
  }
}

```

c. Create a test directory on the device:

```
adb shell mkdir -p /data/local/tmp/adsp
```

d. Push the quantized serialized context binary to the device:

```
adb push <lpai_model>.bin /data/local/tmp/
```

e. Push LPAI libs to the device:

```
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnLpai.so
/data/local/tmp/
```

```
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/
libQnnLpaiStub.so /data/local/tmp/
```

```
adb push    ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/
libQnnLpaiNetRunExtentions.so /data/local/tmp/
```

```
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/
libQnnLpaiSkel.so /data/local/tmp/adsp
```

- f. Push input data, input lists, qnn_config.json and lpai_config.conf to the device:

```
adb push input_list.txt /data/local/tmp/
adb push <input_data>.raw /data/local/tmp/
adb push qnn_config.json /data/local/tmp/
adb push lpai_config.conf /data/local/tmp/
```

- g. Push qnn-net-run to the device:

```
adb push ${QNN_SDK_ROOT}/bin/{QNN_TARGET_ARCH}/qnn-net-run
/data/local/tmp/
```

- h. Set up the environment on the device:

```
adb shell
cd /data/local/tmp/
export LD_LIBRARY_PATH=/data/local/tmp/;/data/local/tmp/adsp
export ADSP_LIBRARY_PATH=/data/local/tmp/adsp
```

- i. Execute the LPAI model using qnn-net-run:

```
./qnn-net-run
--backend libQnnLpai.so
--retrieve_context <lpai_model>.bin
--input_list input_list.txt
--config_file qnn_config.json
```

NOTE: libQnnLpaiSkel.so must be signed by the client. qnn-net-run must be executed with root permissions.

- QNN LPAI native aDSP backend execution via FastRPC – This mode is meant to efficiently execute the model in the LPAI backend by bypassing the inefficiencies associated with data and control transfer via the IPC mechanism. Steps:

- a. Set environments:

```
export QNN_TARGET_ARCH=aarch64-android
export HW_VER=v6
export DSP_ARCH=hexagon-v81
export DSP_VER=V81
```

b. Prepare qnn_config.json and lpai_config.conf files.

• qnn_config.json:

```
{
  "backend_extensions": {
    "shared_library_path": "/data/local/tmp/adsp/libQnnLpaiNetRunExtensions.so",
    "config_file_path": "./lpai_config.conf"
  },
  "context_configs" :{
    "is_persistent_binary" : true
  }
}
```

NOTE: "is_persistent_binary" : true is mandatory for model inference on Audio PD.

• lpai_config.conf:

```
{
  "lpai_backend":
  {
    "target_env": "adsp",
    "enable_hw_ver": "v6"
  },
  "lpai_graph":
  {
    "execute":
    {
      "fps": 30,
      "ftrt_ratio": 10,
      "client_type": "non_real_time",
      "affinity": "soft",
      "core_selection": 0
    }
  }
}
```

c. Create a test directory on the device:

```
adb shell mkdir -p /data/local/tmp/adsp
```

d. Push the quantized serialized context binary to the device:

```
adb push <lpai_model>.bin /data/local/tmp/
```

e. Push LPAI libs to the device:

```
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/libQnnLpai.so
/data/local/tmp/adsp
```

```
adb push ${QNN_SDK_ROOT}/lib/lpai_${HW_VER}/unsigned/
libQnnLpaiNetRunExtensions.so /data/local/tmp/adsp
```

- f. Push input data, input lists, qnn_config.json and lpai_config.conf to the device:

```
adb push input_list.txt /data/local/tmp
adb push <input_data>.raw /data/local/tmp
adb push qnn_config.json /data/local/tmp
adb push lpai_config.conf /data/local/tmp
```

- g. Push qnn-net-run and dependent libs to the device:

```
adb push ${QNN_SDK_ROOT}/bin/{QNN_TARGET_ARCH}/qnn-net-run
/data/local/tmp/

adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/
libQnnNetRunDirect${DSP_VER}Stub.so /data/local/tmp

adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/
libQnnHexagonSkel_App.so /data/local/tmp/adsp

adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/libQnnNetRunDirect$
{DSP_VER}Skel.so /data/local/tmp/adsp
```

- h. Set up the environment on the device:

```
adb shell
cd /data/local/tmp/
export LD_LIBRARY_PATH=/data/local/tmp;/data/local/tmp/adsp
export ADSP_LIBRARY_PATH=/data/local/tmp/adsp
```

- i. Execute the LPAI model using qnn-net-run:

```
./qnn-net-run
--backend adsp/libQnnLpai.so
--direct_mode
--retrieve_context <lpai_model>.bin
--input_list input_list.txt
--config_file qnn_config.json
```

NOTE: All artifacts pushed to /data/local/tmp/adsp must be signed by the client.

NOTE: qnn-net-run must be executed with root permissions

3.2.3 Deployment on target using QNN APIs

Once context binary is generated successfully, use QNN APIs to deploy the model on the target.

- Refer to the `capi_detection_qnn` reference example in the audio add-on for insights on usage of QNN APIs.

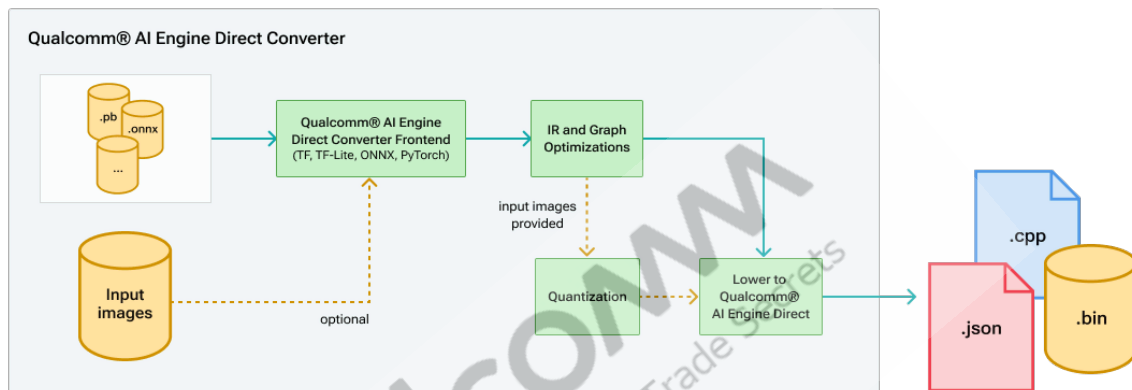
- Clients need to open/link the QNN LPAI BE lib (`libQnnLpai.so`) and QNN system lib (`libQnnSystem.so`) to get access to QNN LPAI BE APIs and QNN system APIs respectively. See Chapter 4 for a list of QNN APIs supported by LPAI BE.

3.3 Model conversion

Users need to convert their custom model into a QNN-compatible format (.cpp and .bin) using QNN converters.

Qualcomm AI Engine Direct currently supports converters for four frameworks: TensorFlow, TFLite, PyTorch, and ONNX.

The following diagram shows the converter workflow.



There are four main parts to each converter:

- Front-end translation – Handles converting the original framework model into the common intermediate representation (IR).
- Common IR code – Contains graph and IR operation definitions as well as various graph optimizations that can be applied to translated graphs.
- Quantizer – Optionally invoked to quantize the model prior to the final lowering to QNN.
- QNN converter backend – Responsible for lowering the IR into the final QNN model API calls.

The converter outputs the following files:

- .cpp source file (for example, model.cpp) containing the required QNN APIs calls to construct a network graph.
- .bin binary file (for example, model.bin) containing network weights and biases as float32 data.
- model_net.json, which is a json format variant to model.cpp.

The following are usages of QNN converter tools:

- Required arguments needed by different converters:

```
qnn-onnx-converter
--input_network <path_to_source_framework_model.onnx>
```

```
qnn-tflite-converter
    --input_network <path_to_source_framework_model.tflite>
```

```
qnn-tensorflow-converter
    --input_network < path_to_source_framework_model.pb>
    --input_dim <"input_node_name">
    <comma_separated_input_dimension_in_NHWC>
    --out_name <output_node_name>
```

```
qnn-pytorch-converter
    --input_network < path_to_source_framework_model.pb>
    --input_dim <"input_node_name">
    <comma_separated_input_dimension_in_NCHW>
```

■ Commonly used arguments (supported by all converters):

- If custom encodings file are needed:

```
--quantization_overrides <custom_encodings.json>
```

- To disable Batchnorm folding:

```
--disable_batchnorm_folding
```

- To specify the input data which represents the model behavior closely (this input will be used by the QNN converter to determine the float range and will be processed by the CPU backend for model quantization):

```
--input_list <path_to_text_file_specifying_input_data>
```

- To preserve IO layout:

```
preserve_io layout <space_separated_list_of_IO_names>
```

- To disable squashing of Relu against convolution based ops for quantized models:

```
--disable_relu_squashing
```


- To specify which quantization schema to use for parameters (weight/bias):

```
--param_quantizer_schema <asymmetric/symmetric/unsignedsymmetric>
```

- To specify the quantization schema to use for activations:

```
--act_quantizer_schema <asymmetric/symmetric/unsignedsymmetric>
```

NOTE: `asymmetric` is the default quantization schema for activations as well as for weight/bias.

- Option to select the bit width to use when quantizing the biases:

```
--bias_bitwidth <8/32>
```

NOTE: The default bias bit width is 8 and 32 is recommended to use for better accuracy.

- Option to select the bit width to use when quantizing the activations:

```
--act_bitwidth <8/16>
```

- Option to select the bit width to use when quantizing the weights:

```
--weights_bitwidth <8/16>
```

- To disable unrolling and expansion of the GRU OP:

```
--multi_time_steps_gru
```

- To disable unrolling and expansion of the LSTM Op

```
--multi_time_steps_lstm
```

- To specify the target environment as LPAI:

```
--target_backend <HTP/LPAI/CPU/GPU/HTA/AIC/DSP/LPAI>
```

- To enable fallback to floating point instead of fixed point:

```
--float_fallback
```

- To override all input dimensions which leverage variable names:

```
--define_symbol batch_size=<batch_size> seq_len=<seq_len>
```

Refer to QNN SDK documentation for more information on QNN converters.

3.4 Model preparation

Model preparation involves two steps:

1. Generation of QNN model library.
2. Generation of QNN context binary for LPAI backend.

3.4.1 Generation of QNN model library

The `qnn-model-lib-generator` is used to generate the QNN model library. The `qnn-model-lib-generator` tool compiles the QNN model source code into artifacts for a specific target.

Usages:

```
qnn-model-lib-generator [-c <QNN_MODEL>.cpp] [-b <QNN_MODEL>.bin] [-t
LIB_TARGETS] [-l LIB_NAME] [-o OUTPUT_DIR]
```

- Required argument(s):

- `-c <QNN_MODEL>.cpp` – File path for the QNN model .cpp file

- Optional argument(s):

- `-b <QNN_MODEL>.bin` – File path for the QNN model .bin file
- `-t LIB_TARGETS` – Specifies the targets to build the models
- `-l LIB_NAME` – Specifies the name to use for libraries
- `-o OUTPUT_DIR` – Path for saving output libraries

The `qnn-model-lib-generator` tool must be selected from the appropriate x86 platform directory located at `{QNN_SDK_ROOT}/bin/x86_64-<Platform>`, where `<Platform>` can be either Windows or Linux.

3.4.2 Generation of QNN context binary for LPAI Backend

The `qnn-context-binary-generator` is used to prepare the QNN context binary for the LPAI backend. The `qnn-context-binary-generator` tool is used to create a context binary by using a particular backend and consuming a model library created by the `qnn-model-lib-generator`.

Usages:

```
qnn-context-binary-generator --model <QNN_MODEL>.so
--backend <QNN_BACKEND>.so
--binary_file <BINARY_FILENAME>
--config_file <CONFIG_FILE>.json
--backend_binary <eai_model>
```

- Required argument(s):

- `--model <QNN_MODEL>.so` – Path to QNN model library file
- `--backend <QNN_BACKEND>.so` – Path to a QNN backend .so library (libQnnLpai.so)
- `--binary_file <BINARY_FILENAME>` – Name of the binary file to save the context binary to with a .bin file extension. If not provided, no backend binary is created.

- `--config_file <CONFIG_FILE>.json` – Path to a JSON config file related to backend extensions and context priority.
- **Optional argument(s):**
 - `--backend_binary <eai_model>` – Name of the binary file to save a backend specific context binary to with .bin file extension. If not provided, no backend binary is created.

NOTE: The `qnn-context-binary-generator` tool must be selected from the appropriate x86 platform directory located at `${QNN_SDK_ROOT}/bin/x86_64-<Platform>`, where `<Platform>` can be either Windows or Linux.

The JSON config file is meant for `qnn-context-binary-generator` to get details of LPAI backend extensions.

The following are the contents of the json file. The `is_persistent_binary` flag should be set to TRUE when the model is preparing for ADSP.

```
{
  "backend_extensions":
  {
    "shared_library_path": "path_to_libQnnLpaiNetRunExtensions.so",
    "config_file_path": "path_to_lpai_params.conf"
  },
  "context_configs":
  {
    "is_persistent_binary": true
  }
}
```

NOTE: `libQnnLpaiNetRunExtensions.so` should be picked from `${QNN_SDK_ROOT}/bin/x86_64-<Platform>` where `<Platform>` could be Windows or Linux.

`lpai_params.conf` is the configuration file that contains LPAI backend specific details. This file has two major fields:

- **LPAI backend details, used by `qnn-context-binary-generator` during offline preparation.**
 - `target_env` – Specifies the target environment.
 - `enable_hw_ver` – Indicates the eNPU hardware version.
- **LPAI graph details, containing two fields:**
 - Graph preparation parameters, which are used by `qnn-context-binary-generator` during offline preparation.
 - Graph execution parameters, which are used by `qnn-net-run` during execution.

The following are the contents of the Conf file. The `enable_hw_ver` flag specifies the eNPU core version. For SM8850, this version is v6.

```
{
  "lpai_backend": {
    "target_env": "adsp",
    "enable_hw_ver": "v6"
  },
  "lpai_graph": {
    "prepare": {
      "enable_batchnorm_fold": true,
      "exclude_io": false
    },
    "execute": {
      "fps": 1,
      "ftrt_ratio": 10,
      "client_type": "real_time",
      "affinity": "soft",
      "core_selection": 0,
      "mem_type": "ddr",
    }
  }
}
```

In the above sample `lpai_params.conf` file, all graph execution parameters are set to the default values. Refer to QNN SDK documentation for more information.

4 QNN API Descriptions and Call Sequences

From SM8850 onwards, eAI runtime has been migrated to QNN runtime (also called QNN-LPAI backend).

The following table contains the list of QNN APIs supported for the LPAI backend and their descriptions.

QNN API	Description
<code>QnnContext_createFromBinary</code>	Create a context from a stored binary.
<code>QnnBackend_create</code>	Initialize a backend library and create a backend handle.
<code>QnnGraph_setConfig</code>	Set/modify configuration options on an already created graph.
<code>QnnGraph_getProperty</code>	Retrieve a list of graph properties
<code>QnnGraph_finalize</code>	Finalize the graph.
<code>QnnGraph_execute</code>	Synchronously execute a finalized graph.
<code>QnnContext_free</code>	Free the context and all associated graphs, operations, and tensors
<code>QnnGraph_retrieve</code>	Retrieve a graph based on name.
<code>QnnContext_setConfig</code>	Set/modify configuration options on an already generated context.
<code>QnnBackend_free</code>	Free all resources associated with a backend handle.
<code>QnnBackend_getApiVersion</code>	Get the QNN API version.
<code>QnnBackend_getBuildId</code>	Get build ID for backend library.
<code>QnnProperty_hasCapability</code>	Query the capability of the backend.
<code>QnnProfile_create</code>	Create a handle to a profile object.
<code>QnnProfile_getEvents</code>	Get QNN profile events collected on the profile handle.
<code>QnnProfile_getSubEvents</code>	Get QNN profile event handles nested within the QNN profile event handle.
<code>QnnProfile_getEventData</code>	Query the data associated with QNN profile event.
<code>QnnProfile_getExtendedEventData</code>	Query the data associated with QNN profile extended event.
<code>QnnProfile_free</code>	Free memory associated with the profile handle. All associated <code>QnnProfile_EventId_t</code> event handles are implicitly freed.
<code>QnnDevice_getPlatformInfo</code>	Get the collection of devices and cores that a QNN backend can recognize and communicate with.
<code>QnnDevice_freePlatformInfo</code>	Free memory allocated during <code>QnnDevice_getPlatformInfo()</code>
<code>QnnDevice_create</code>	Create a logical device to handle a subset of hardware resources available on the platform.
<code>QnnDevice_setConfig</code>	Free the created device and perform any deallocation of the resources allocated during device create.
<code>QnnDevice_getInfo</code>	Get platform info associated with a device handle.
<code>QnnDevice_free</code>	Free the created device and perform any deallocation of the resources allocated during device create.

QNN API	Description
<code>QnnMem_deRegister</code>	Deregister a memory handle which was registered via <code>QnnMem_register</code> and invalidates <code>memHandle</code> for the given backend handle.
<code>QnnMem_register</code>	Register existing memory to memory handle. Used to instruct QNN to use this memory directly.
<code>QnnLog_create</code>	Create a handle to a logger object. This function can be called before <code>QnnBackend_create()</code> .
<code>QnnLog_setLogLevel</code>	Set the log level for the supplied log handle.
<code>QnnLog_free</code>	Free the memory associated with the log handle

The following table contains the list of QNN System APIs and their descriptions.

QNN APIs	Description
<code>QnnSystemContext_create</code>	Create an instance of the QNN system context.
<code>QnnSystemContext_free</code>	Free the instance of the System Context object and clears any intermediate memory allocated and associated with a valid handle.
<code>QnnSystemContext_getBinaryInfo</code>	Get context info from the serialized binary buffer.

For full list of QNN APIs, refer to QNN SDK documentation `<QNN_SDK_DIR>/docs/QNN/api-rst/unabridged_api.html`.

4.1 QNN APIs usage guidelines

- `QnnInterface` is used to access QNN APIs.
- The QNN Interface is an abstraction that combines all component APIs and provides a single unified interface to the user.
- This provides a major convenience to users to operate with a fixed set of API hooks through the interface instead of relying on dynamic symbol lookup using `dlsym()` calls.
- The QNN Interface API is made available to users using what are referred to as API providers.
- API provider list can be obtained via `QnnInterface_getProviders()`.
- Usages:

```
const QnnInterface_t **providerlist = NULL;
uint32_t num_providers = 0;

QnnError_Error_t result = QnnInterface_getProviders(&providerlist,
                                                    &num_providers);
```

4.2 Data structures and enumerations

4.2.1 QnnContext_Config_t

The QnnContext_Config_t structure provides context configuration.

This data structure is defined in `QnnContext.h` header file present at `<QNN_SDK_DIR>/include/QNN/`.

Parameters	Descriptions
QnnContext_ConfigOption_t option	Provides option to set context configs See 4.2.2 for details.
uint8_t isPersistentBinary	Used with QNN_CONTEXT_CONFIG_PERSISTENT_BINARY

4.2.2 QnnContext_ConfigOption_t

This enum defines context config options.

This enum has multiple options, but the following option is specific to QNN-LPAI BE. This enum is defined in `QnnContext.h` header file present at `<QNN_SDK_DIR>/include/QNN/`.

Property	Descriptions
QNN_CONTEXT_CONFIG_PERSISTENT_BINARY	Indicates that the context binary pointer is available during <code>QnnContext_createFromBinary</code> and until <code>QnnContext_free</code> is called.

4.2.3 QnnBackend_Property_t

This structure provides backend property.

This data structure is defined in `QnnBackend.h` header file present at `<QNN_SDK_DIR>/include/QNN/`.

Parameters	Descriptions
QnnBackend_PropertyOption_t option	Option is used by clients to set or get any backend property.
QnnBackend_CustomProperty_t customProperty	Pointer to the backend property requested by client.

4.2.4 QnnLpaiDevice_DeviceInfoExtension_t

`QnnDevice_getPlatformInfo ()` uses this structure to list the supported device features/information.

This data structure is defined in `QnnLpaiDevice.h` header file present at `<QNN_SDK_DIR>/include/QNN/LPAI/`.

Parameters	Descriptions
uint32_t socModel	An enum value defined in Qnn Header that represents SoC model
uint32_t arch	It shows the architecture of the device
const char* domainName	It shows the domain name of the device

4.2.5 QnnLpaiGraph_Mem_t

QnnGraph_setConfig() API used this structure to set custom configs for scratch and persistent buffer.

This data structure is defined in QnnLpaiGraph.h header file present at <QNN_SDK_DIR>/include/QNN/LPAI.

Parameters	Descriptions
QnnLpaiMem_MemType_t memType	An enum value defined in enum QnnLpaiMem_MemType_t to memory type of buffer.
uint32_t size	Size of buffer
void* addr	Pointer to buffer

4.2.6 QnnLpaiMem_MemType_t

This enum contains memory type supported by LPAI backend.

This enum is defined in QnnLpaiMem.h header file present at <QNN_SDK_DIR>/include/QNN/LPAI.

Property	Descriptions
QNN_LPAI_MEM_TYPE_DDR	Main memory, only available in non-island mode
QNN_LPAI_MEM_TYPE_LLC	Last level cache
QNN_LPAI_MEM_TYPE_TCM	Tightly coupled memory for hardware
QNN_LPAI_MEM_TYPE_UNDEFINED	Unused

4.2.7 QnnLpaiGraph_SetConfigOption_t

This enum contains custom configs for LPAI backend graph.

This enum is defined in QnnLpaiGraph.h header file present at <QNN_SDK_DIR>/include/QNN/LPAI.

Property	Descriptions
QNN_LPAI_GRAPH_SET_CFG_SCRATCH_MEM	Used to set scratch memory configs. Struct: QnnLpaiGraph_Mem_t
QNN_LPAI_GRAPH_SET_CFG_PERSISTENT_MEM	Used to set persistent memory configs. Struct: QnnLpaiGraph_Mem_t
QNN_LPAI_GRAPH_SET_CFG_PERF_CFG	Used to set custom client perf configs. Struct: QnnLpaiGraph_PerfCfg_t
QNN_LPAI_GRAPH_SET_CFG_CORE_AFFINITY	Used to set core affinity configs. Struct: QnnLpaiGraph_CoreAffinity_t
QNN_LPAI_GRAPH_SET_CFG_UNDEFINED	Unused

4.2.8 QnnGraph_Config_t

This structure provides graph configuration.

This data structure is declared in `QnnGraph.h` header file present at `<QNN_SDK_DIR>/include/QNN/`.

Parameters	Descriptions
QnnGraph_ConfigOption_t option	An enum value defined in enum <code>QnnGraph_ConfigOption_t</code> to set custom graph configs.
QnnGraph_CustomConfig_t customConfig	Pointer to custom graph configs

4.2.9 QnnLpaiGraph_CustomConfig_t

This structure is used by `QnnGraph_setConfig` to set backend specific configurations before finalizing the graph.

This data structure is declared in `QnnLpaiGraph.h` header file present at `<QNN_SDK_DIR>/include/QNN/LPAI/`.

Parameters	Descriptions
uint32_t option	An enum value defined in enum <code>QnnLpaiGraph_SetConfigOption_t</code> to set backend specific configs to graph.
void* config	Pointer to custom configs

4.2.10 QnnLpaiBackend_BufferAlignmentReq_t

This structure contains parameters needed to align the start address of buffer and size of buffer.

This data structure is declared in `QnnLpaiBackend.h` header file present at `<QNN_SDK_DIR>/include/QNN/LPAI/`.

Parameters	Descriptions
uint32_t startAddrAlignment	Represents start address alignment of buffer. The start address of the buffer must be startAddrAlignment-byte aligned
uint32_t sizeAlignment	Represents buffer size alignment. The allocated buffer must be a multiple of sizeAlignment bytes

4.2.11 QnnLpaiBackend_GetPropertyOption_t

This enum contains the set of properties supported by the LPAI backend. Objects of this type are to be referenced through `QnnBackend_CustomProperty_t`

This enum is defined in `QnnLpaiBackend.h` header file present at `<QNN_SDK_DIR>/include/QNN/LPAI/`.

Property	Descriptions
QNN_LPAI_BACKEND_GET_PROP_ALIGNMENT_REQ	Used to get the start address alignment and size alignment requirement of buffers. Struct: <code>QnnLpaiBackend_BufferAlignmentReq_t</code>

Property	Descriptions
QNN_LPAI_BACKEND_GET_PROP_REQUIRE_PERSISTENT_BINARY	Used to query if cached binary buffer needs to be persistent until QnnContext_free is called. If yes, then need to specify QNN_CONTEXT_CONFIG_PERSISTENT_BINARY during QnnContext_createFromBinary
QNN_LPAI_BACKEND_GET_PROP_UNDEFINED	Unused

4.2.12 QnnLpaiGraph_CustomProperty_t

This structure is used by QnnGraph_getProperty() API to get backend specific configurations.

This data structure is defined in QnnLpaiGraph.h header file present at

<QNN_SDK_DIR>/include/QNN/LPAI/.

Parameters	Descriptions
uint32_t option	An enum value defined in enum QnnLpaiGraph_GetPropertyOption_t to retrieve backend specific property.
void* property	Pointer to custom property

4.2.13 QnnLpaiGraph_CoreAffinity_t

This structure is used by QnnGraph_getProperty to get backend specific configurations.

This data structure is defined in QnnLpaiGraph.h header file present at

<QNN_SDK_DIR>/include/QNN/LPAI/.

Parameters	Descriptions
QnnLpaiGraph_CoreAffinityType_t affinity	Used to set the affinity of selected eNPU core enum: QnnLpaiGraph_CoreAffinityType_t
uint32_t coreSelection	eNPU core; 0 = little core, 1 = big core

4.2.14 QnnLpaiGraph_CoreAffinityType_t

This enum contains the possible set of affinities supported by eNPU HW.

This enum is defined in QnnLpaiGraph.h header file present at

<QNN_SDK_DIR>/include/QNN/LPAI/.

Property	Descriptions
QNN_LPAI_GRAPH_CORE_AFFINITY_SOFT	Used to set affinity to soft. Struct: QnnLpaiGraph_CoreAffinity_t
QNN_LPAI_GRAPH_CORE_AFFINITY_HARD	Used to set affinity to hard. Struct: QnnLpaiGraph_CoreAffinity_t
QNN_LPAI_GRAPH_CORE_AFFINITY_UNDEFINED	Unused

4.2.15 QnnLpaiGraph_PerfCfg_t

This structure is used to set Client's performance requirement for eNPU Usage. User can configure it before finalizing the graph.

This data structure is declared in `QnnLpaiGraph.h` header file present at `<QNN_SDK_DIR>/include/QNN/LPAI/`.

Parameters	Descriptions
uint32_t fps	Used to set frame per second (fps)
uint32_t ftrtRatio	Used to set FTRT ratio
QnnLpaiGraph_ClientPerfType_t clientType	Used to set client type (Real time or Non-real time) enum: QnnLpaiGraph_ClientPerfType_t

4.2.16 QnnLpaiGraph_ClientPerfType_t

This enum contains the type of client which can be configured by user before finalizing the graph.

This data structure is declared in `QnnLpaiGraph.h` header file present at `<QNN_SDK_DIR>/include/QNN/LPAI/`.

Property	Descriptions
QNN_LPAI_GRAPH_CLIENT_PERF_TYPE_REAL_TIME	Used to set client as REAL TIME Struct: QnnLpaiGraph_PerfCfg_t
QNN_LPAI_GRAPH_CLIENT_PERF_TYPE_NON_REAL_TIME	Used to set client as NON-REAL TIME Struct: Used to set client as REAL TIME
QNN_LPAI_GRAPH_CLIENT_PERF_TYPE_UNDEFINED	Unused

4.2.17 Qnn_ApiVersion_t

This structure is used to provide the versions of both the core QNN API and any Backend Specific API.

This data structure is declared in `QnnTypes.h` header file present at `<QNN_SDK_DIR>/include/QNN/`.

Parameters	Descriptions
Qnn_Version_t coreApiVersion	Used to query version of the QNN core API common to all backends
Qnn_Version_t backendApiVersion	Used to query version of the backend-specific API

4.2.18 Qnn_Version_t

This structure is used to provide a version number using 3 values: major, minor, patch.

This data structure is declared in `QnnTypes.h` header file present at `<QNN_SDK_DIR>/include/QNN/`.

Parameters	Descriptions
uint32_t major	Represents major version
uint32_t minor	Represents minor version
uint32_t patch	Represents patch version

4.3 QNN APIs Call Flow

QNN model integration involves three steps:

1. Initialization
2. Execution
3. Deinitialization

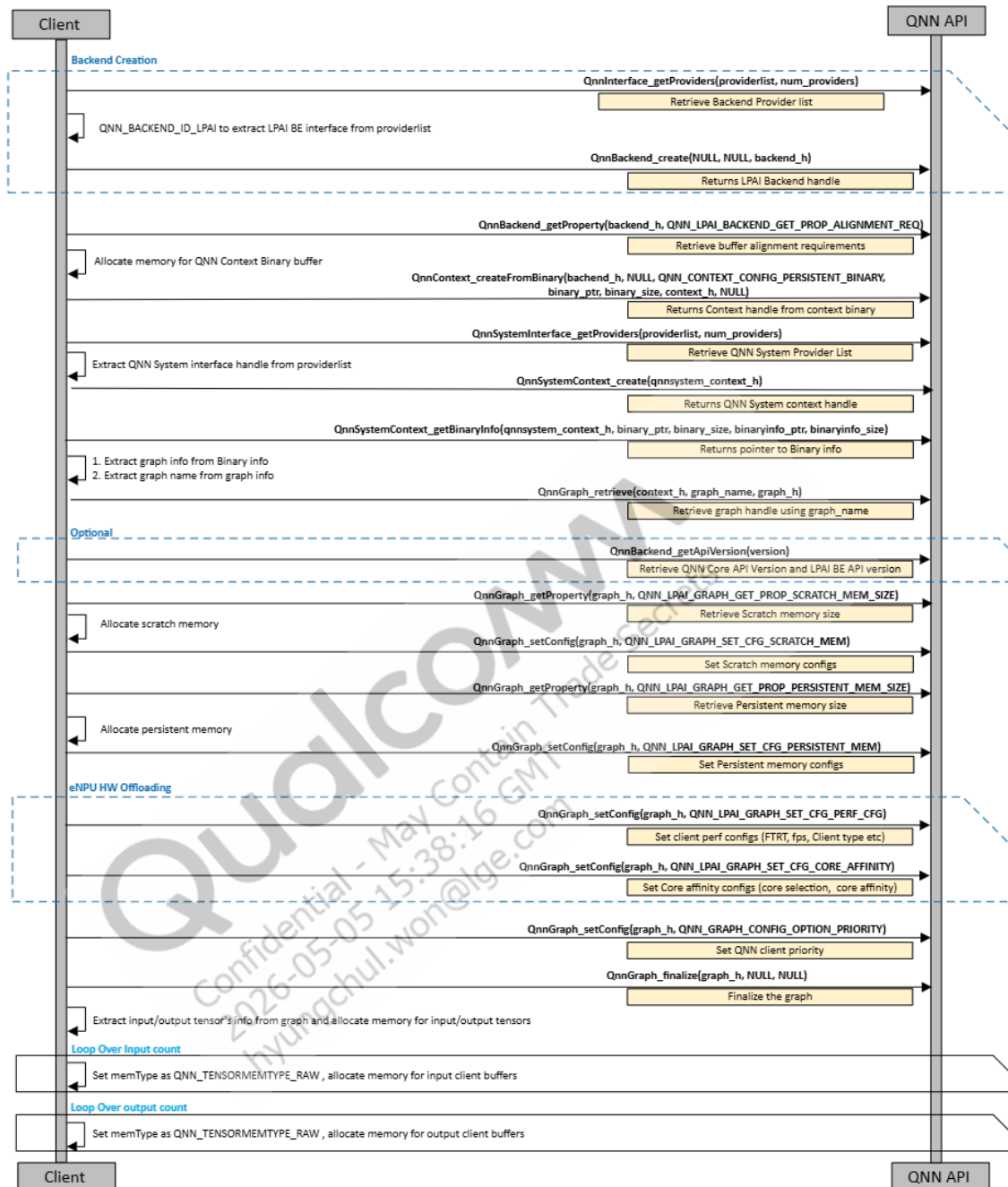
4.3.1 Initialization

Initialization consists of the following steps:

1. Extract interface –Retrieve the LPAI BE interface and QNN System interface.
 - LPAI BE interface:
 - The LPAI BE interface is essential for calling QNN APIs that are supported by LPAI BE. It acts as an interface with QNN functionalities provided by LPAI BE.
 - `QnnInterface_getProviders()` API is used to extract the list of all backend supported by QNN.
 - Users need to extract LPAI BE interface from the extracted provider list using backend identifier `QNN_LPAI_BACKEND_ID`.
 - QNN system interface:
 - Used to call QNN System APIs. It provides context for the QNN system, enabling the user to manage and execute various system-level operations.
 - `QnnSystemInterface_getProviders()` API is used to extract QNN system interface.
2. Create handles –Create handles for the LPAI BE and QNN system context.
 - LPAI BE handle – `QnnBackend_create()` API is used to create LPAI BE handle.
 - QNN system context handle – `QnnSystemContext_create()` API is used to create QNN system context handle.
3. Query buffer alignment requirements –Query the memory alignment requirements using the `QNN_LPAI_BACKEND_GET_PROP_ALIGNMENT_REQ` property.
 - Start address alignment – Query the alignment requirements for the start address of the buffer. This ensures that the buffer starts at an address that meets the alignment criteria specified by the LPAI BE.

- Buffer size alignment: Similarly, query the alignment requirements for the buffer size. This ensures that the size of the buffer adheres to the alignment constraints, which is crucial for efficient memory access and performance.
- 4. Allocate memory for QNN context binary buffer – Select the appropriate memory pool (Island or Non-Island) based on use case requirements, ensuring the memory meets alignment criteria before allocation.
- 5. Create the QNN context using the context binary buffer – The content of a context created in this way cannot be further altered. This context handle is used to retrieve any model related information.
- 6. Retrieve graph information and graph name – Use the QNN System API `QnnSystemContext_getBinaryInfo()` to get binary information and retrieve the graph name from the binary information.
- 7. Retrieve graph using graph name – Once the graph name is retrieved, retrieve the graph using graph name.
- 8. Allocate scratch and persistent memory – To allocate scratch and persistent memory, query scratch and persistent memory size requirements using the QNN graph set API `QnnGraph_getProperty()` with properties `QNN_LPAI_GRAPH_GET_PROP_SCRATCH_MEM_SIZE` and `QNN_LPAI_GRAPH_GET_PROP_PERSISTENT_MEM_SIZE` respectively.
- 9. Set scratch and persistent memory configurations into the graph – To set scratch and persistent memory configurations into graph, use `QnnGraph_setConfig()` with the properties `QNN_LPAI_GRAPH_SET_CFG_SCRATCH_MEM` and `QNN_LPAI_GRAPH_SET_CFG_PERSISTENT_MEM` respectively.
- 10. Set client perf and eNPU core affinity – The client can use the `QnnGraph_setConfig()` API with the properties `QNN_LPAI_GRAPH_SET_CFG_PERF_CFG` and `QNN_LPAI_GRAPH_SET_CFG_CORE_AFFINITY` to set client perf configs and select the required core and affinity respectively.

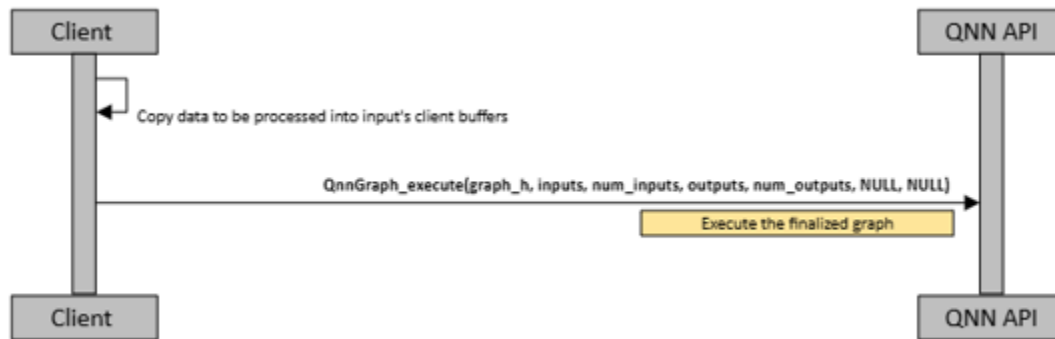
The following diagram shows the initialization call flow.



11. Set/modify the QNN client priority – Set/modify the priority of the QNN instance using the `QnnGraph_setConfig()` API with the property `QNN_GRAPH_CONFIG_OPTION_PRIORITY`.
12. Once all configurations are done, use `QnnGraph_finalize()` to finalize the graph.
13. Allocate input and output tensors – After the graph is finalized, retrieve input and output tensors from the graph and allocate memory for them.

Set the memType of each input and output tensor as `QNN_TENSORMEMTYPE_RAW` and allocate the client buffers. After initialization, the graph is ready to be executed.

4.3.2 Execution



Before executing the finalized graph, copy the data to be processed into input client buffers.

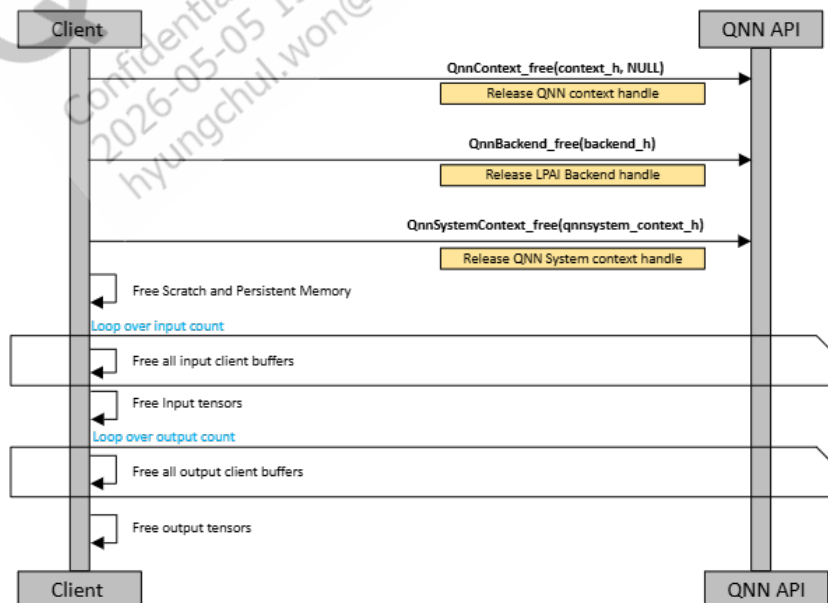
The `QnnGraph_execute()` API is used to call the finalized graph, which will execute the model on the eNPU hardware and provide the output in output client buffers after processing.

4.3.3 Deinitialization

Deinitialization consists of the following steps:

1. Release the QNN context handle.
2. Release the LPAI backend handle.
3. Release the QNN system context handle.
4. Free scratch and persistent memory.
5. Free input and output tensors and associated client buffers.

The following diagram shows the deinitialization call flow.



5 Reference example

5.1 Build and Run CAPI based QNN example on Target (ADSP)

This section describes the compilation and integration steps needed to run a CAPI-based QNN example on SM8850.

5.1.1 Compiling CAPI example in Hexagon SDK

Install the required tools as described in Section 2. The following commands can be used to compile the example.

- Island mode with eNPU:

```
make hexagon BUILD=Debug DSP_ARCH=v79 STATIC=FALSE TREE=1 LPI=1  
USE_ENPU=1
```

- Island mode without eNPU:

```
make hexagon BUILD=Debug DSP_ARCH=v79 STATIC=FALSE TREE=1 LPI=1  
USE_ENPU=0
```

- Non-Island mode with eNPU:

```
make hexagon BUILD=Debug DSP_ARCH=v79 STATIC=FALSE TREE=1 LPI=0  
USE_ENPU=1
```

- Non-Island mode without eNPU:

```
make hexagon BUILD=Debug DSP_ARCH=v79 STATIC=FALSE TREE=1 LPI=0  
USE_ENPU=0
```

Once compilation is completed successfully, `capi_detection_qnn.so` will be present at `<EXAMPLE_FOLDER>/hexagon_Debug_toolv19_v79/ship/`

NOTE: Users need to sign QNN system libs and QNN backend libs using the `elfsigner` script present in the Hexagon SDK. Once signed, these `.so` files, along with the CAPI module, should be pushed to the target device.

5.1.2 Build the SoundTriggerTestApp from the LA build

The SoundTriggerTestApp is present in the LA build and can be compiled via the following commands:

```
cd <LA_UM_ROOT_DIR>
source build/envsetup.sh
lunch -userdebug
cd frameworks/base/tests/SoundTriggerTestApp/
mm
```

Once the compilation is completed successfully, SoundTriggerTestApp.apk should be present at <LA_UM_ROOT_DIR>/out/target/product/canoe/system/priv-app/SoundTriggerTestApp/.

5.1.3 Update resourcemanager.xml file and ACDB files

To update ACDB files and resourcemanager.xml to run custom Voice UI in Non-LPI/LPI mode, refer to *Low Power Island Voice Activation – SVA 9.0* (80-64995-153).

5.2 CAPI example overview

This section explains the QNN- related code for an example detection qnn module.

5.2.1 Data structures and enumerations

To make various function calls from the CAPI wrapper, some data structures must be initialized.

- In module_qnn.h file, the qnn_data_t data structure stores all parameters needed for the QNN model and its input/output tensor.
- In this example, it assumes that the QNN model has one input and output tensor, but this can be changed via the MAX_MODEL_INOUT_COUNT macro.

5.2.2 Initialization

In module_qnn.c, the module_qnn_init() wrapper function is used to create and initialize QNN-LPAI instance. This function does the following:

- Retrieves the LPAI backend interface and QNN system interface with the helper functions module_qnn_get_lpai_interface() and module_qnn_get_qnn_system_interface() respectively.
- Creates the LPAI BE handle.
- Queries buffer alignment requirements and stores their values in the qnn_data_t data structure using the helper function module_qnn_get_buffer_alignment_requirement().

module_qnn_init() function can be called from the CAPI module's init function callback.

Once the LPAI interface (`lpai_interface`) is retrieved successfully, it can be used to call QNN APIs supported for LPAI-BE with the help of LPAI BE handle (`backend`).

The following helper functions are used to initialize/load/store the QNN-LPAI instance:

- `module_qnn_store_and_load_model()` – Stores and loads a new QNN model with the helper function `module_qnn_store_model()` and `module_qnn_load_model()` respectively. It also unloads the previous model if any model is already loaded.
- `module_qnn_store_model()` – Stores a new QNN model and allocates model memory and copies the model in `qnn_data_t` data structure.
- `module_qnn_load_model()` – Loads a new QNN model and does the following:
 - Creates context handle from serialized binary using `QnnContext_createFromBinary()` and creates QNN system context using `QnnSystemContext_create()`.
 - Retrieves serialized binary buffer info using `QnnSystemContext_getBinaryInfo()`.
 - Stores `graph_info` and `graph_name` into `qnn_data_t` data structure.
 - Retrieves graph handle (`graph_h`) using `QnnGraph_retrieve()` via `graph_name`.
 - Queries QNN API version and LPAI BE API version with the help of helper function `module_qnn_get_model_info()`.
 - Queries scratch and persistent memory requirements using helper functions `module_qnn_get_scratch_mem_size()` and `module_qnn_get_persistent_mem_size()` respectively and allocates the required scratch and persistent memory.
 - Sets configuration such as `MEM_TYPE`, size, and pointers pointing to the memory for scratch and persistent memory using helper functions `module_qnn_set_scratch_config()` and `module_qnn_set_persistent_config()`.
 - If eNPU hardware offloading is requested by the client, then it sets client perf configs using the helper function `module_qnn_set_client_perf_config()` and sets core affinity with the help of `module_qnn_set_core_affinity()`.
 - Optionally, clients can also modify priority of QNN-LPAI Instance with the helper function `module_qnn_set_priority_config()`.
 - After setting all the configurations, it finalizes the graph via `QnnGraph_finalize()`.
 - Prepares input/output tensors with the helper functions `module_qnn_update_inout_tensors()`.
 - If anything goes wrong while storing/loading the model, this function unloads the model using helper function `module_qnn_unload_model()` followed by releases the model using helper function `module_qnn_release_model()`.
- `module_qnn_update_inout_tensors()` – Retrieves input/output tensor info from the graph and allocates them using `module_qnn_allocate_tensors()`. On failure, it frees all related memory in `qnn_data_t` using `reset_input_output_info()`.
- `module_qnn_allocate_tensors()` – Used to allocate QNN input/output tensors with `QNN_TENSORMEMTYPE_RAW` and calculates their size using helper function `calculate_tensor_size()`. It allocates memory for client buffers needed for LPAI BE. On failure, it resets tensors using `reset_input_output_info()`.
- `reset_input_output_info()` – Releases all input/output tensor related memory allocated in the `qnn_data_t` data structure.

5.2.3 Execute

The function `module_qnn_execute()` explains the execute call flow. This function does the following:

- Copies input data from the input pointer to the `p_qnn_data` input pointer
- Calls `QnnGraph_execute()` for processing.

This function should be called repeatedly from the CAPI module's process function call.

5.2.4 Deinitialization

The functions `module_qnn_unload_and_release_model()` and `module_qnn_deinit()` explain the deinitialization call flow.

- `module_qnn_unload_and_release_model()` - Unloads and releases and unloads the loaded QNN model using helper functions `module_qnn_release_model()` and `module_qnn_unload_model()` respectively.
 - This function can be called from the CAPI module's end callback function. After calling this function, the user must call `module_qnn_deinit()` function.
- `module_qnn_deinit()` - Clears `qnn_data_t` data structure.
 - This function can be called from the CAPI module's end callback function only after the QNN model is unloaded and released successfully by calling `module_qnn_unload_and_release_model()`.

6 FAQs

How do I create handles for QNN components?

Use `QnnBackend_create()` to create the LPAI BE handle and `QnnSystemContext_create()` to create the QNN system handle.

What happens if `QnnInterface_getProviders()` returns zero providers?

This typically indicates that the backend libraries are not correctly loaded or the environment is not set up properly.

Ensure the following:

- The backend .so files are present on target.
- `LD_LIBRARY_PATH` includes the directory containing QNN backend libraries.

Why is buffer alignment important?

To ensure efficient memory access, start address alignment and buffer size, alignment must be queried using the `QnnBackend_getProperty()` with `QNN_LPAI_BACKEND_GET_PROP_ALIGNMENT_REQ`.

What if memory alignment requirements are not met?

Failure to meet alignment requirements can lead to:

- Crashes
- Performance degradation
- Backend API errors

Always query and alignment constraints before memory allocation.

Can multiple backends be used simultaneously?

No, QNN only supports initializing and using one backend at a time per context. You must create separate contexts for different backends.

Can graphs be modified after finalizing the context?

No. Once the context is created from a binary, it becomes immutable. Any changes in a graph require regenerating the context binary.

Is `QnnGraph_execute()` a blocking call?

Yes. `QnnGraph_execute()` is blocking, meaning it will not return until the model execution is complete.

Where is the output stored after execution?

Output is written to the output client buffers that were allocated and configured during initialization.

Is the order of deinitialization important?

Yes. Always release resources in the reverse order of their allocation to avoid dependency issues or memory access violations.

Can the same context be reused after deinitialization?

No. Once a context is released, it must be re-created from scratch for further use.

A References

6.2 Related documents

Title	Number
Qualcomm Technologies, Inc (QTI)	
AI Engine Direct SDK	80-63442-50
EAI Integration in CAPI Modules for Audio Reach	80-VN500-31
SM8850/SM8850P AI/ML Software Overview	80-74889-65
SM8850/SM8850P Linux Android Audio Overview	80-74889-50
Low Power Island Voice Activation – SVA 9.0	80-64995-153
CAPI Custom Module Integration into Audio Reach Signal Processing Framework	80-VN500-17

6.3 Acronyms and terms

Acronym or term	Definition
QNN	Qualcomm Neural Network
LPAI	Low power AI subsystem
eNPU	Embedded Neuron Processing Unit
eAI	Embedded Artificial Intelligence
BE	Backend
CAPI	Common Audio Processor Interface
ADSP	Audio Digital Signal Processor
SPF	Signal Processing Framework
LPI	Low power Island
TCM	Tightly Coupled Memory
DDR	External Memory
QACT	Qualcomm Audio Calibration Tool
ACDB	Audio Calibration Data Base

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("Qualcomm Technologies"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "Qualcomm Internal Use Only", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.